OrthoVis

TechLauncher 2025
State of the Project

Version History			
Version	***Date***	***Description***	***Contributor***
1.0	22/10/2025	Document completed outlining the current state of the OrthoVis 2.0 project. This document contains information on all features implemented by the team as well as recommendations for further development.	2025 Development Team

Table of Contents

Overview

2025 was the first year that OrthoVis had entered Techlauncher. As such, the 2025 team was tasked with building from scratch. This included understanding the underlying inputs and outputs of the application, as well as the different components of the application. These are known as 'modules' and collectively they are the very crux of the application.

This document outlines development of the OrthoVis application by modules. Specially each sections aims to explain how the module should operate, the achievements in 2025 by the development team, and finally the development opportunity in each module for those who join the project after 2025.

Backend

Module Objective

This module serves as the interface between the frontend GUI and all other modules. All the user interactions at the frontend will go through the backend and afterwards to the modules such as segmentation, registration, etc. The backend contains the project metadata, classes and the design patterns that facilitate the program flow between the frontend and the core modules.

Current work

As it stands, there are two main parts in the backend module.

1. The project meta data – When a new project is created, the data of the project is represented by the data.json file under the Projects directory.

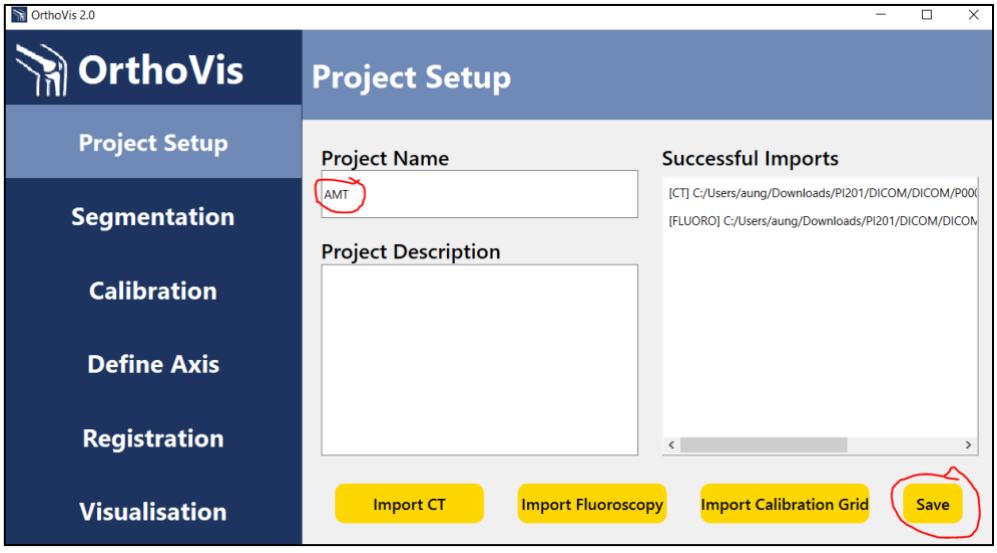

Image 1 - Project Setup, Naming and Imports

Image 2 - Project Metadata Format

2. Design patterns (Located in objects.py)

    1. The singleton pattern - This is to model the project data mentioned above and to make sure that only one instance of the project data is present throughout the lifetime of the program. Reusing this singleton instance across different modules, the project data can be accessed. Accessing the project data and updating it will be done through this singleton instance.

    2. The state pattern - This pattern is for keeping track of what state the project data is in. When we first create a project, we can see that the state is in “RawState” in the screenshot above. After Segmentation, the state will be updated to “SegementState”. Each state has three functions handle_save, handle_process, handle_to_string. These functions will behave differently depending on the state of the project data.

    3. The factory pattern - This pattern is responsible for turning the string representation of state into an object. This is needed when opening an existing project. The meta data data.json is read when the project is opened. The factory pattern will read the state string from data.json and turn it into state object. Afterwards, the handle_save function will behave according to the state object.

    The testing of the design patterns are done in the Test folder unit_tests.py

    To run the tests, make sure you are in the root directory of OrthoVis (E.g. C:\Users\aung\Documents\Comp4500\OrthoVis)

    Then run the command, python -m Test.unit_tests

For more information, the UML class diagrams are available here

Further Works & Known Bugs

Currently, the state design pattern is modelled based on the flow of the program. You are free to add in new states or modify the existing ones or even completely replace it if you have a better way of representing the state of the project.

The current design assumes the sequential flow of the states and the GUI is also designed accordingly.

Module 1 – Project Setup

Module Objective

This module is responsible for creating new projects. With reference to the screenshots in the section above, there is a form at the front end. The form asks for the name, description, and the file imports. After completing the form, a new folder will be created under the "Projects" folder along with the meta data json file.

Achievements in 2025

- Importing data. Every import is marked with a label. For example, the first import as the "[CT]" label since we import it by clicking the "Import CT" button. If we import by clicking "Import Fluoroscopy" button, we can see "[FLUORO]" label in the second row.

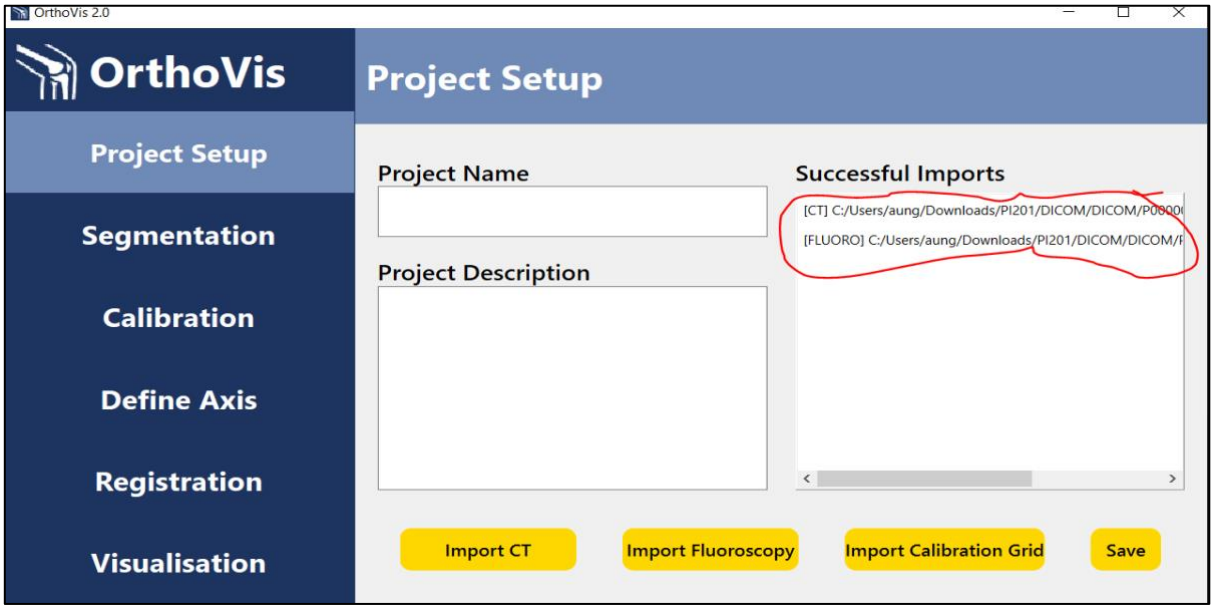

Image 3 - Project Setup

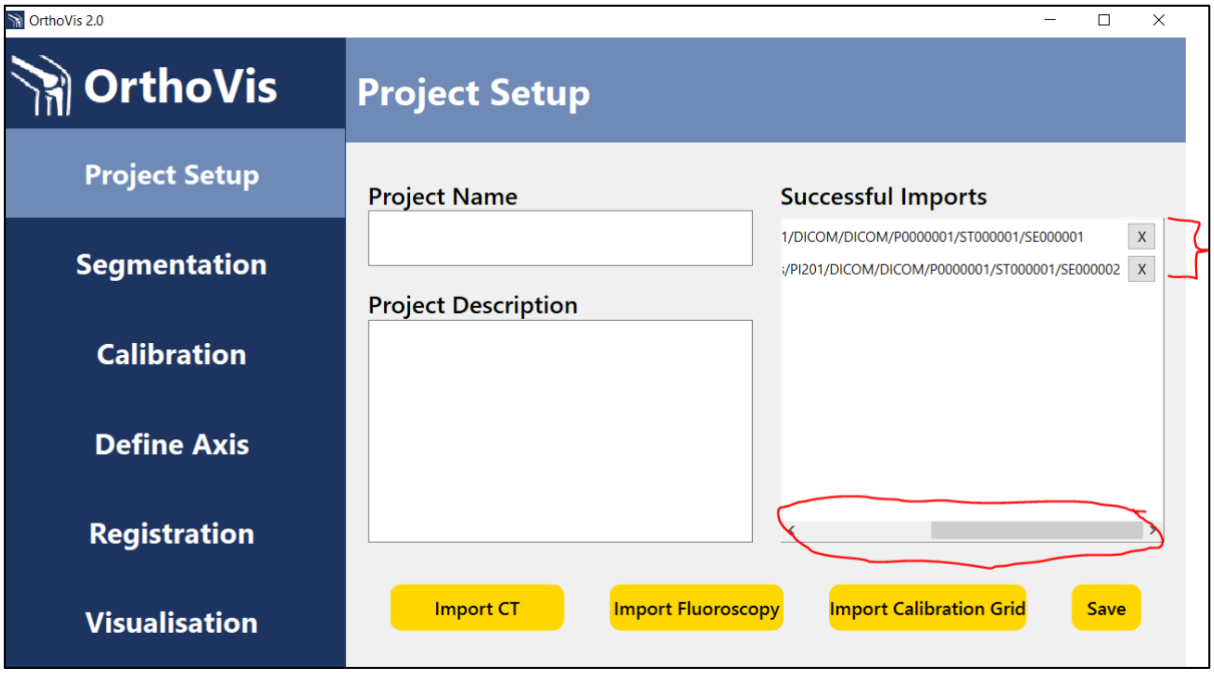

Image 4 - How to Delete Imports

The wrong imports can be deleted by sliding the bottom scroll to reveal the delete buttons. Clicking the "X" will delete the entry.

Further Works & Known Bugs

Currently, the file import only allows the one CT, one fluoroscopy and one calibration grid. In the future, we might need to import multiple CTs, fluoroscopy's and calibration grids. To do so, the path_dict data structure needs some modifications. At the moment, this data structure gets populated as the following if we make the following imports

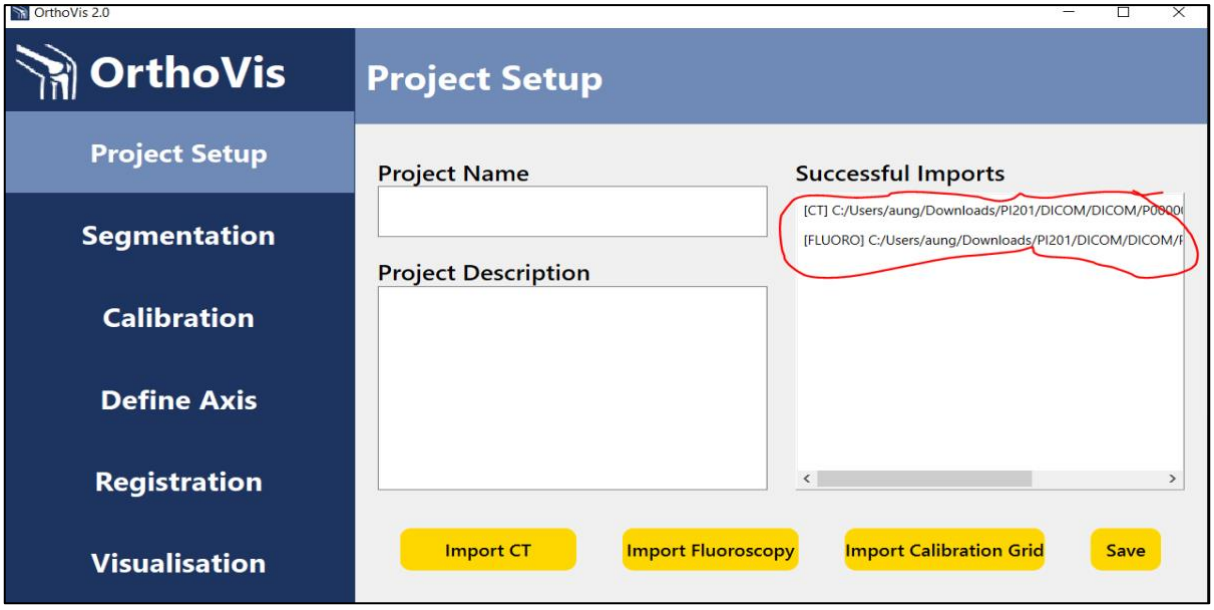

Image 5 - Successful Imports

Image 6 - Terminal Outputs After Import

Currently the dictionary looks like this after imports:

{'CT': 'C:/Users/aung/Downloads/PI201/DICOM/DICOM/P0000001/ST000001/SE000001',
'fluoro': 'C:/Users/aung/Downloads/PI201/DICOM/DICOM/P0000001/ST000001/SE000002'}

Every entry has one string as its value.

Instead, we will have something like

{'CT': ['C:/Users/aung/…', 'C:/Users/aung/…', 'C:/Users/aung/…'],

'fluoro': ['C:/Users/aung/…', 'C:/Users/aung/…', 'C:/Users/aung/…']}

Every entry will have a list of strings as its value.

Hence, the object model of the patient and meta data json also needs to be updated accordingly.

```python
class Patient:
    def __init__(self, name: str, description: str, CT: str, fluoro: str, caligrid: str):
        self.name = name
        self.description = description
        self.CT = CT
        self.fluoro = fluoro
        self.caligrid = caligrid
        self._seg_masks_dir = None  # Added for segmentation masks directory


    @property
    def name(self):
        return self._name

    @name.setter
    def name(self, value):
```

Image 7 - Patient Data Type

Instead of being single strings, the CT, fluroro and caligrid fields will be changed to string lists.

Module 2 – Segmentation

Module Objective

The objective of the segmentation module is to provide a fully automated and efficient workflow for isolating specific bones from 3D CT scans (in DICOM or Nifti format). This module serves as the initial and critical step in the OrthoVis pipeline. It leverages a powerful model, TotalSegmentator, to produce accurate and precise segmentations. The module must also provide a robust visualization interface allowing the user to view the original CT data and the resulting segmentation masks in standard axial, coronal, and sagittal planes, complete with interactive tools for inspection, mask modification, and verification. A key performance requirement is that the entire automated segmentation and post-processing workflow should be completed in under 30 minutes as per clients' expectations.

Achievements in 2025

- **Automated Segmentation Backend:** A proof-of-concept backend was successfully established, which calls the pre-trained TotalSegmentator model to perform fully automated segmentation of regions of interest. Post-processing via Otsu refinement have been implemented to refine the model's output, ensuring high accuracy and precision. The entire pipeline has been optimized, far exceeding meet the target, to completion under 10 minutes.
- **Advanced Visual Rendering:** A sophisticated visualization component was developed using an embedded VTK widget. It can load and render DICOM or NIfTI files in the three standard views (axial, coronal, and sagittal). The renderer includes a suite of interactive graphical features (implemented with keyboard and mouse shortcuts):
    - In Browse mode (pre-segmentation), the user can:
        - Inspect the original CT files with a slider that scroll through all CT slices (*mouse scroll*)
        - Zoom in/out the current CT slice (*Ctrl + scroll*)
        - Pan across a CT slice (*Ctrl + Shift + left drag*)
        - Select bones of interest for segmentation from the ROI dropdown menu
    - In Edit mode (post-segmentation), segmentation masks are automatically overlaid onto the CT images. The user can toggle individual masks to disable/enable visibility, and using keyboard/mouse shortcuts:
        - Add or remove pixels from the selected mask to refine precision and accuracy of mask (Add: *Left click + drag*; Erase: *Ctrl + left click + drag*)
        - Adjust brush size of the area to be added/removed (*+/- on keyboard*)
        - Save edits made to masks (*S on keyboard*)

- **Application Framework and Integration:** The frontend modules and backend code have been integrated to form a cohesive, working application skeleton, demonstrating a clear path from data loading to segmented output visualization. When satisfied with changes, the user can proceed to the next calibration module by clicking on the "Proceed to Calibration" button; as a safeguard, all modifications to masks will be automatically saved before switching modules.

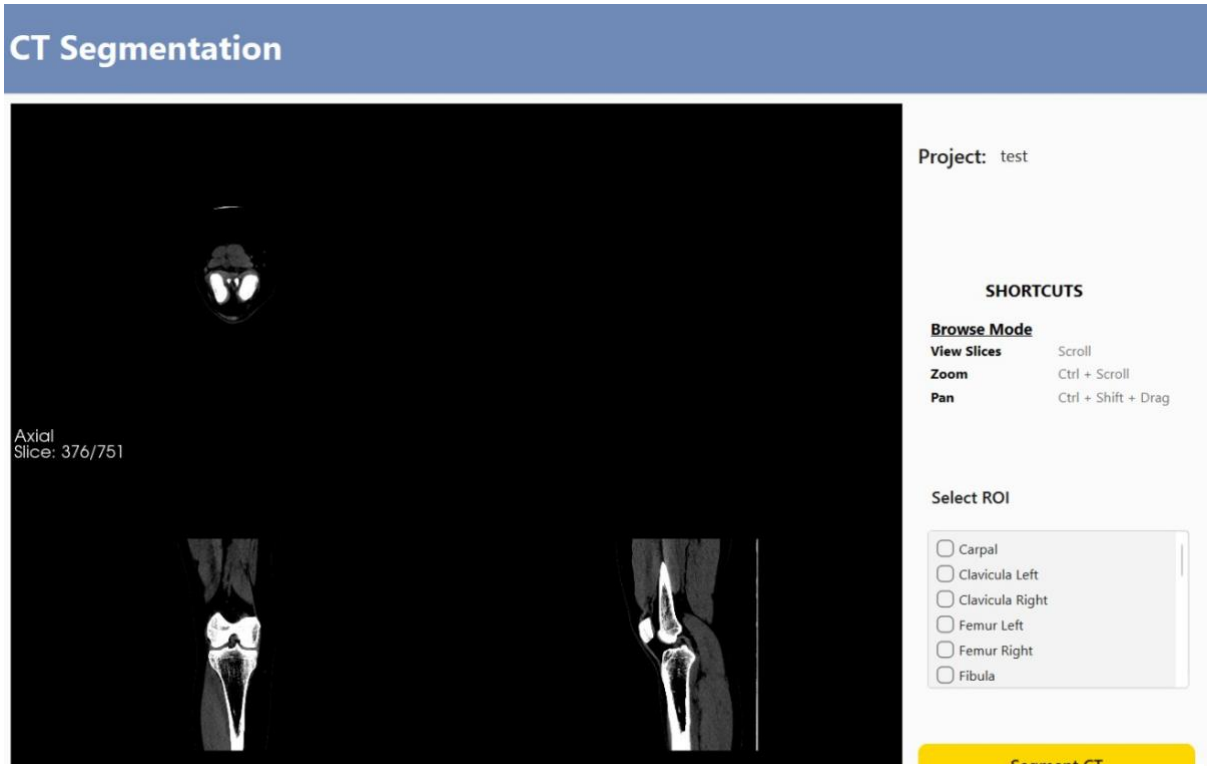

Image 10 - Edit mode (post-segmentation) GUI: in the post-segmentation VTK widget, the user may toggle between "Browse" and "Edit" modes by left-clicking the "Mode: Browse/Edit" text area. Loss of precision and accuracy in auto-segmentation may be manually resolved by adding or removing pixels in the corresponding mask with our brush feature, which is activated only in "Mode: Edit" mode. The brush area may be increased/decreased with the brush size (keyboard shortcuts: +/-) to align with the magnitude of the desired modification. At any time during the modification process, the user may elect to manually save changes with the "Save Edits" button or by pressing the S key on the keyboard. A safeguard has been placed to automatically save the latest versions of masks before proceeding to the next calibration module, even if the user forgets to save.

Further Works & Known Bugs

- Known Bugs:
  - In some instances, **clicking the "Segment CT" button does not go through the first try, but succeeds on the second. This is likely due a bug in the linking code between the GUI button and backend and requires fixing.**

    - In very large or high-resolution DICOM series, the initial loading and rendering time can be slow, impacting user experience. Segmentation may take more than 10 minutes systems with lower graphical processing power.
        - Please add a command for TotalSegmentator to recognise GPUs on macOS. As of version 1.0, GPUs with *cuda* compatibility are successfully recognised on Windows/Linux, but not Apple's M(x) GPUs. Running TotalSegmentator on macOS has thus relied on CPUs, significantly slower than its theoretical performance.
- Future work:
    - Module complete – no new features or functionality are needed.
    - Fix bugs aforementioned and test for a smooth user experience.

Module 3 – Calibration

Module Objective

The Calibration module is designed to provide accurate geometric correction for fluoroscopy and X-ray images using an AutoAlign grid-based method. It aims to eliminate image distortion caused by projection

There's a section in this paper that might be really helpful to understand the technicality - https://onlinelibrary.wiley.com/doi/full/10.1002/jor.21003. But to explain this in simpler words – Flouro images are warped, the way the beams land on an object creates distortion (which means a cube isn't a proper cube anymore). So we need to fix this distortion, and the way we do that is by taking a fluoroscopy shot of a calibration cube (or square grid) with beads placed on it in the front (red beads) and back (blue beads). You detect where those beads land in the distorted image and fit a mapping (an overlay grid) that pulls them back to their true evenly spaced positions. This mapping/corrected distance is then used to apply to every frame of your flouro to undistort it and use is correctly for the purposes of projection.

Achievements in 2025

- Completed core **AutoAlign v0.3** algorithm, supporting two-layer grid alignment

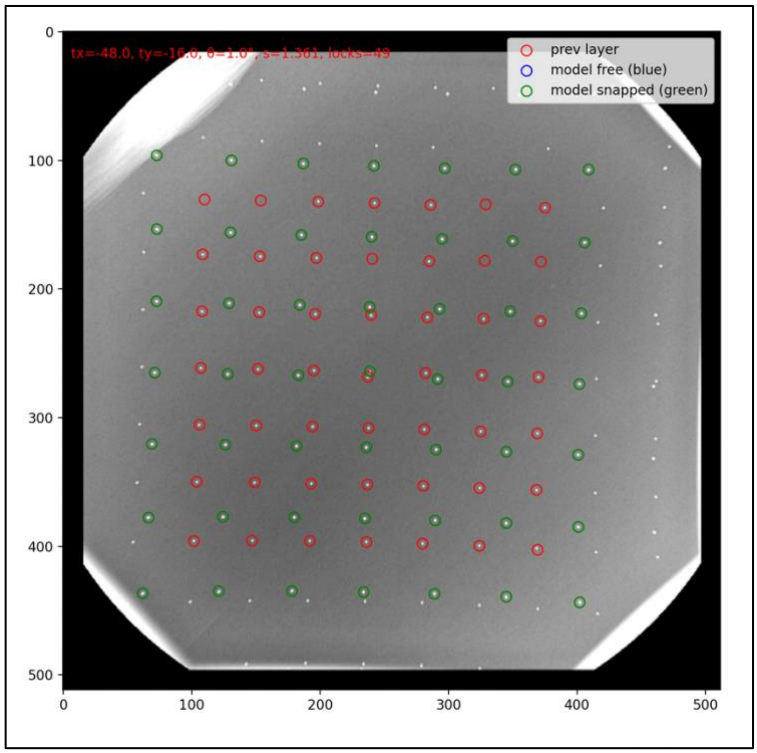

Image 11 - Layer 0

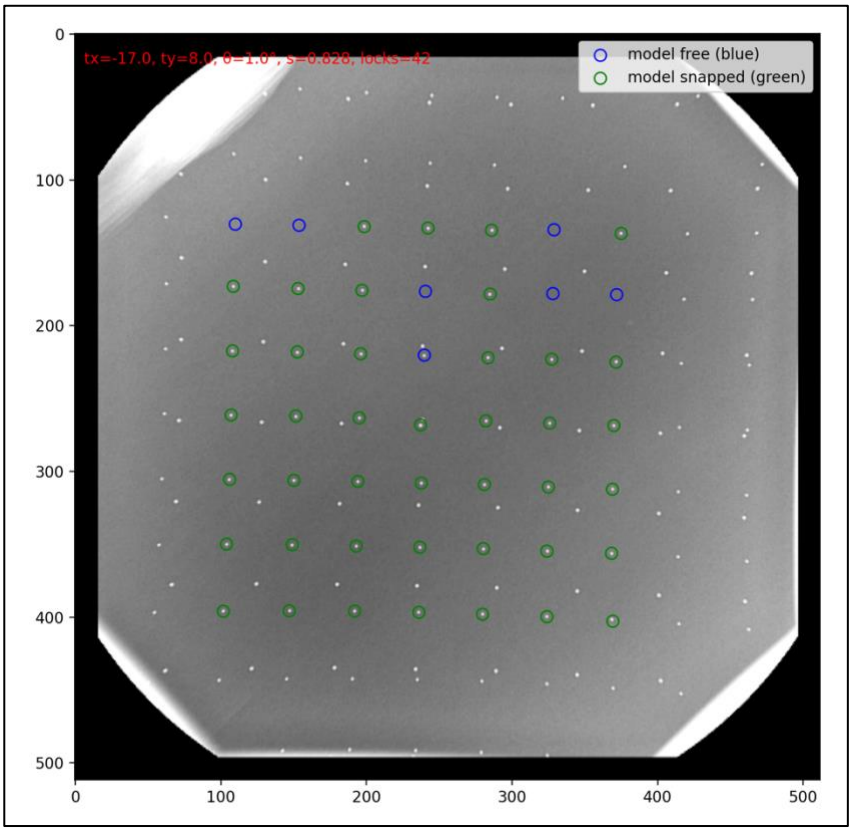

Image 12 - Layer 1

- Implemented **automatic DICOM reading**, bead detection (detect_candidates), manual point addition, and snapping logic (interactive_manual_snap).

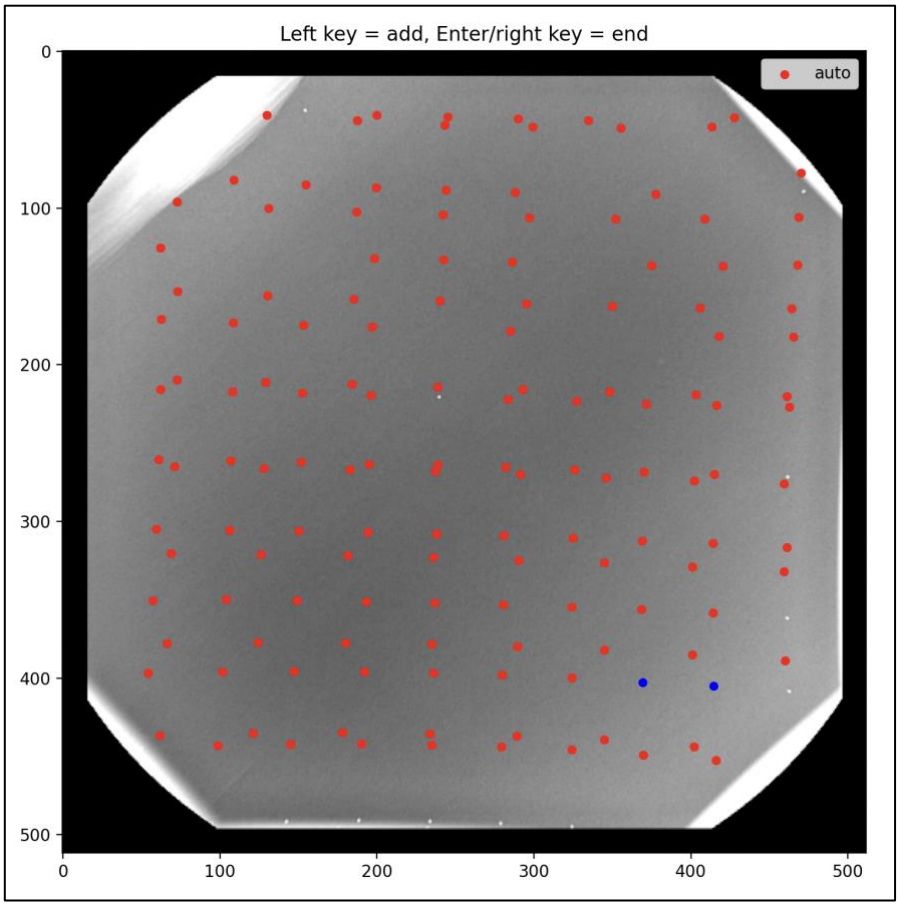

Image 13 - Bead Detection

- Verified geometric accuracy through interactive alignment tests on calibration phantoms.
- Save beads and export for layer0, layer1, and merged dist_data.mat, enabling further distortion modeling.

Further Works & Known Bugs

- Integrate the AutoAlign module into the **PySide6 main GUI** (replace or extend the existing Calibration tab).
- Connect all GUI buttons (alignLayer0Button, alignLayer1Button) and enable real-time feedback in QGraphicsView.
- Implement automatic saving confirmation and progress display.
- Plan to extend to 3-layer calibration for curved detectors or cone-beam geometry.

Known Bugs

- GUI integration not completed yet — current version runs as a standalone script.

- Matplotlib window still required for manual alignment (pending Qt event migration).
- Bead snapping threshold may mis-trigger on dense regions.
- .mat overwrite warning missing when re-running calibration.

Module 4 – Define Axes

Module Objective

For quantitative analysis of movement of bones relative to each-other after registration is complete, landmarks on each bone need to be decided upon, thus defining axes that the movement of each bone can be compared on.

Achievements in 2025

This module was not implemented by the 2025 team. However, the GUI contains a window for defining axes module to be implemented, ready for further development of the module.

Further Works & Known Bugs

The module was not started by the 2025 team. While not strictly necessary for 2D-3D registration, *Defining Axes* is crucial for obtaining quantitative data on the bones within a joint move with respect to each other in each movement pattern. Liaise with Joe and Catherine for demonstration and understanding.

Module 5 – Registration

Module Objective

The objective of the registration module is to register a flattened 2D image of a segmented bone (from the CT) to the fluoroscopy itself. For this module to work as desired, the bone image needs to be manipulated with 6 Degrees of Freedom (DoF). This is, the bone can be translated along the X, Y and Z axis, as well as rotating on these axes. Rotation on the Z axis is essentially 'spinning' the bone in place, however rotation on the X and Y axis are more troublesome. These are called out-of-plane rotations, and when rotating out of plane, the 2D image will need to be recomputed. With these rotations you are essentially wanting to view the bone from a different angle, so the flattening needs to be recomputed. For the flattened image to be accurate and correct, a projective transform must be applied during the flattening process. This transform is derived from the calibration module.

When the user is happy with the relative positioning of the flattened bone image over the corresponding bone in the fluoroscopy window, the bone should be registered using a back-end AI model. The next bone will then need to be loaded into the frame. This should be done for all bones required for registration on the given joint/movement.

The OrthoVis 2025 team conducted analysis of the original OrthoVis registration pipeline which can be found at this link. This paper is helpful to understand concepts of projective and out-of-plane transforms.

Achievements in 2025

The 2025 team has been able to implement a large amount of the registration module. On transition, the module will load the first frame of the fluoroscopy series into a VTK window which is able to be cycled through using the prev/next buttons.

By pressing the 'load bone mask' button, the user is able to load the flattened femur image. This takes a couple of seconds as the flattening needs to occur first. The flattening process is handled by the following files in the registration directory:

- registration_window.py
- edge_map.py

The process first fetches the corresponding bone mask from current project's meta-data and uses this to extract the region of interest from the CT. From here, a summation is applied to the extracted region, creating a 2D weighted array. Edge detection algorithms (Canny and Sobel) are then used to create a silhouette type image with is loaded on the fluoroscopy frame see through ***Image 14*** and ***Image 15*** below. When the flattened image has been loaded into the VTK window, the user is able to translate with 3 DoF and rotate with 3 DoF – however there are some bugs, see below.

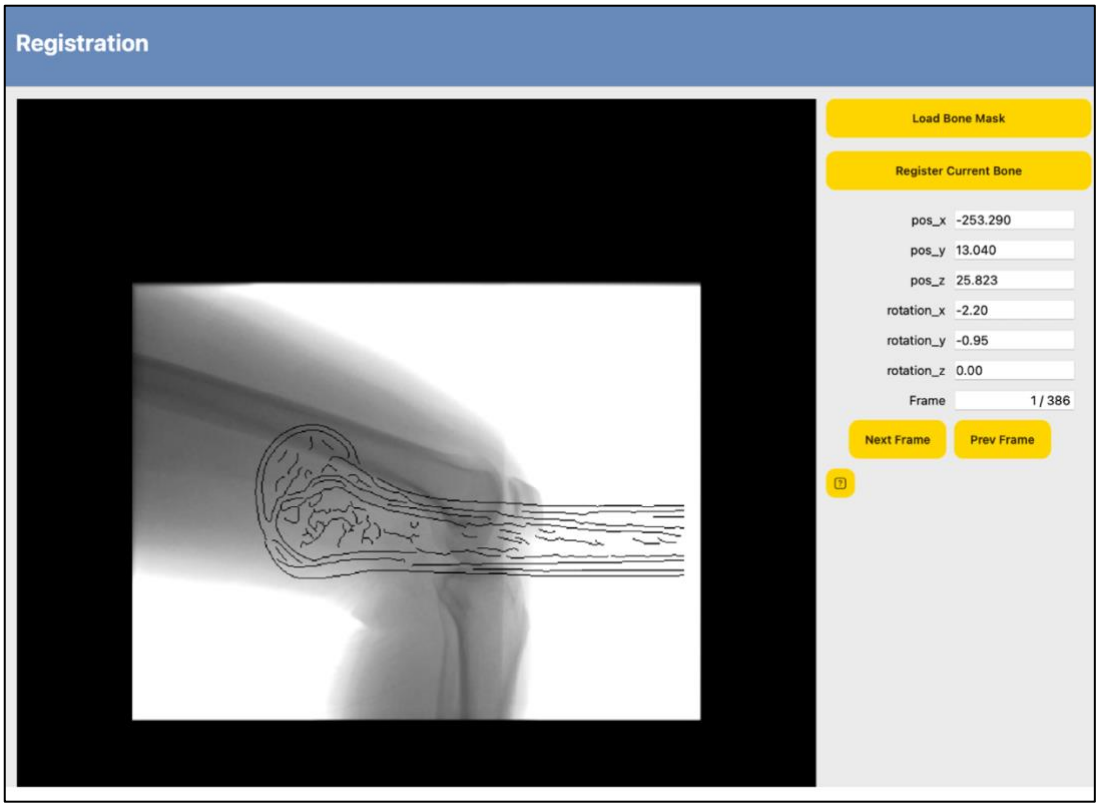

Image 14 - Loaded Bone-Mask Pre-Alignment

Image 15 - Bone Mask Post-Alignment

Further Works & Known Bugs

Several features need to be further developed to complete this module.

1. **Sequential loading of flattened bone images**. This feature still needs to be fully implemented. At current, the registration module is hardcoded to only load a flattened femur, which is assuming that the user is working with a knee, and has already segmented the femur.

   In the case of a knee, a femur should be loaded first, followed by the tibia and fibula.

2. **Connecting registration back-end model to this module**. The 2025 team could not get as far as connecting any sort of registration back-end model to the front even manual registration. The idea is that, when the user is satisfied with the relative position of the 2D flattened bone image over the fluoroscopy, the register button should be clicked, starting a back-end ML script to register the bone across the rest of the fluoroscopy sequence. This is a crucial component to the end output of the OrthoVis application. Suggest working closely with Mark Pickering to understand how and why this needs to be done.
3. **Pipeline for flattening and loading a 2D image needs to be refined**. The current pipeline for flattening and displaying 2D bone mask projections requires optimisation. This issue is closely related to point (1) above. When a bone mask is loaded and the user performs an out-of-plane rotation (X or Y), the system re-extracts the region of interest from the full CT volume and then re-flattens it. This process is slow, unstable, and occasionally buggy. A more efficient approach would involve caching or saving the extracted CT region locally within the project before flattening. Subsequent recomputations could then reuse the cached data, avoiding the repeated extraction step and significantly improving performance and reliability.

Group Reflections

Below outlines a series of group reflections throughout the development of OrthoVis 2.0 in 2025. The 2025 team feels these reflections are an important learning opportunity for future developers working on the project.

1) ***Don't rush into programming straight away***. The 2025 team has spent a long time developing a wide foundation of knowledge in both process and technical understanding required to be a successful team from both a course and a stakeholder perspective. This is fully documented in the project GitHub repository. Read all the project resources thoroughly, understand each module in the application and the science which backs it. Utilise Mark Pickering for technical understanding and lean on Joe Lynch and Catherine Galvin as much as you need. Develop a team charter and group expectations, spend time to get to know your team members, you are going to be spending the whole year together!

2) ***Be realistic about what you can achieve in each sprint***. If 2026 follows the same structure as 2025, you will only have three days (3 x 8h) of working time per sprint. The team needs to make sure sprint goals and deliverables are achievable in this time frame and ensure stakeholder expectations are managed properly.

3) ***Maintain proper git etiquette, including correct branching and merging practice***. Make sure to divide feature development into difference branches. Always pull when opening the project, conduct rigorous code review (merge with reviewers). Make sure to document progress on PBI tickets through agile tool (GitHub projects recommended).

4) ***Suggestion: work in sub-teams or utilise pair programming, divide and conquer and avoid taking on too much work individually***. Delegation is key to succeeding as a team.

5) ***Remember: TechLauncher is a learning experience, mistakes happen***. The most important thing is this program gives you as a team, and as individuals, many opportunities to learn from your mistakes and become the highest achievers you can be.

Conclusion

While this document has outlined a number of directions for development in each module, there is also scope for continued development on the GUI, ensuring a clean, modern and easy to use interface. Future developers should also look at the flow of the application, specifically moving forwards and backwards between modules, addressing how this works, the sequence actions should be undertaken in.